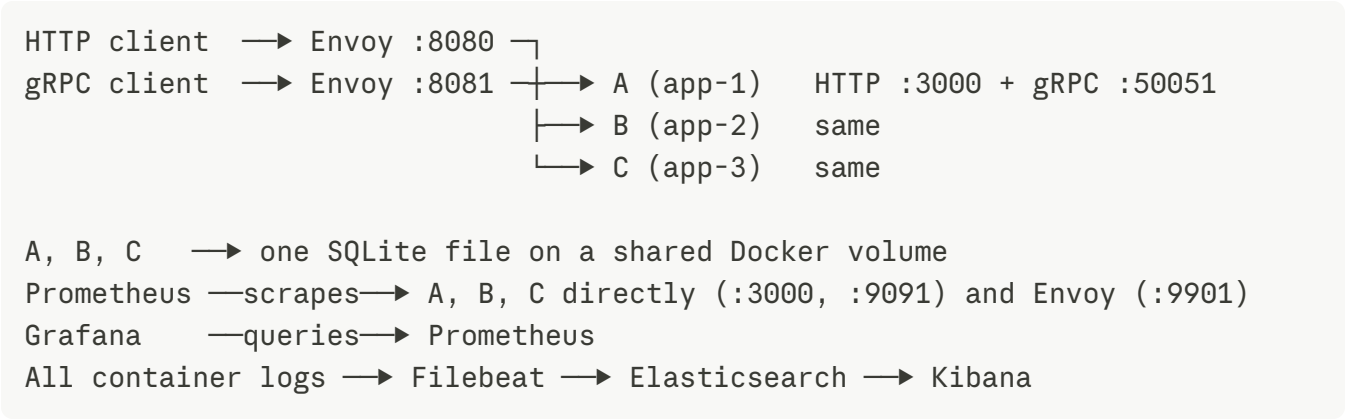
A3 — Results and Conclusions (draft)

Architecture



- Three replicas from `docker compose up --scale app=3`. Each container runs both servers in one Node process (`start.js`).
- Each replica is identified by its hostname (`replica` in logs and metrics). Prometheus adds a stable `replica_name` label: A, B, C.
- The client sends a `trace-id` header (HTTP) or metadata (gRPC). Envoy logs it, and the replica logs the same value.

Shared state

State	Where it lives	Shared?	What happens with 3 replicas
Data (generations)	SQLite on a Docker volume	**Shared**	Every replica reads and writes the same data.
Idempotency records	Same SQLite table (a repeated id returns `created: false`)	**Shared**	A duplicate is rejected whichever replica receives it.

State	Where it lives	Shared?	What happens with 3 replicas
Rate-limit counter	Each replica's memory (5 requests per 10 s)	**Not shared**	Each replica counts separately, so under round robin a client can make up to 15 requests per 10 s, and whether it gets a 429 depends on which replica it hits. Fix: keep the counter in a shared store such as Redis (not implemented).

Metrics and queries

Prometheus scrapes every **5 s**. Every rate, share and percentile uses a **30 s window**. Grafana dashboard: "A3 - Load balancing per replica" (exported JSON in the repo).

Panel	Query (HTTP; gRPC uses the gRPC metrics)
Request rate	`sum by (replica_name) (rate(http_requests_total{route!="/metrics"}[30s]))`
Share of traffic	the rate above ÷ the total rate × 100
Active requests	`sum by (replica_name) (http_requests_in_flight)`
p50 / p95	`histogram_quantile(0.95, sum by (le, replica_name) (rate(http_request_duration_seconds_bucket[30s])))`
Errors by status	`sum by (replica_name, status) (rate(http_requests_total{status!~"2.."}[30s]))`

Notes:

- Each panel adds `or 0 * up{...}`, so an idle or stopped replica shows **0** instead of disappearing.

- Latency buckets run from 1 ms to 2.5 s; percentiles are estimates from these buckets.
- gRPC errors use the gRPC status label, because a failed gRPC call still returns HTTP 200.
- HTTP active requests include the `/metrics` scrape itself (about 1).
- No unique IDs are used as metric labels; `replica_name` has only 3 values.

Summary of all runs

Workload for every run: one HTTP client (`GET /api/generations`) and one gRPC client (`SaveGeneration` , fixed id) started together, each **20 req/s, open-loop**, 60 s (90 s for failure/recovery). No client retries, no circuit breakers. Shares come from Envoy's access logs (counts per upstream host); client p95 from `loadtest.py` .

Run / interface	Policy and fault	Target / actual req/s	A / B / C traffic shares	Client p95	Error rate	Explanation
Baseline / HTTP	A only	20 / 20.0	100 / 0 / 0 %	362 ms	0 %	Only A is in Envoy's cluster; B and C run idle.
Baseline / gRPC	A only	20 / 20.0	100 / 0 / 0 %	406 ms	0 %	As above.
C / HTTP	Round robin, equal	20 / 20.0	33.4 / 33.3 / 33.3 %	290 ms	0 %	Per-request rotation: 401 / 400 / 399.
C / gRPC	Round robin, equal	20 / 20.0	33.3 / 33.3 / 33.3 %	228 ms	0 %	Per-call rotation over one client connection (HTTP/2 listener): 400 / 400 / 400.
D1 / HTTP	Weighted RR 2:1:1	20 / 20.0	50.0 / 24.9 / 25.0 %	225 ms	0 %	Matches the 50 / 25 / 25 target.
D1 / gRPC	Weighted RR 2:1:1	20 / 20.0	50.0 / 25.0 / 25.0 %	133 ms	0 %	600 / 300 / 300. Weight changes who gets traffic, not capacity.
D2 / HTTP	RR, B +200 ms	20 / 20.0	33.4 / 33.3 / 33.3 %	230 ms	0 %	RR ignores speed: 33.3 % of requests slow (Envoy p80 202 ms).
D2 / gRPC	RR, B +200 ms	20 / 20.0	33.3 / 33.3 / 33.3 %	211 ms	0 %	Same: 33.3 % slow (Envoy p80 204 ms).
D2 / HTTP	Least request, B +200 ms	20 / 20.0	39.8 / 18.8 / 41.4 %	256 ms	0 %	Slow requests fall to 18.8 % (Envoy p80 5 ms).

Run / interface	Policy and fault	Target / actual req/s	A / B / C traffic shares	Client p95	Error rate	Explanation
D2 / gRPC	Least request, B +200 ms	20 / 20.0	39.3 / 18.6 / 42.2 %	212 ms	0 %	Slow requests fall to 18.6 % (Envoy p80 7 ms).
D3 / HTTP	RR + health checks, B stopped then restarted	20 / 20.0	33 / 33 / 33 → 50 / 0 / 50 → 33 / 33 / 33 %	100 ms (OK) / 218 ms (all)	3.3 %	53 × 503 + 6 timeouts, all in the ~9 s before B was removed.
D3 / gRPC	RR + health checks, B stopped then restarted	20 / 20.0	33 / 33 / 33 → 50 / 0 / 50 → 33 / 33 / 33 %	53 ms (OK) / 142 ms (all)	3.3 %	58 × UNAVAILABLE + 1 DEADLINE_EXCEEDED, same window.

Replica IPs used to read Envoy's logs (from `:9901/clusters`): A = app-1, B = app-2, C = app-3. IPs change on rebuild; the mapping was re-checked before each experiment.

Distribution checked against Envoy's access logs

For the Part C round-robin run, Envoy's JSON access log (counted per `interface` and `upstream_host`) shows:

Interface	A	B	C	Total
HTTP	401	400	399	1200
gRPC	400	400	400	1200

This matches the client's 1200 requests per interface and the dashboard's ~33 % per replica. The dashboard briefly showed 36 / 32 / 32 % for HTTP; the logs show this was a 30 s-window artefact at the start of the run, not real imbalance.

The failure run agrees in the same way: clients counted 118 failures (59 + 59) and Envoy logged 117–118 error responses in the same seconds.

Proxy and backend log examples

Envoy's JSON access logs go to Elasticsearch through Filebeat. One Kibana search per interface returns the Envoy event and the replica's event for the same request.

HTTP: `app.trace_id : "kibana-http-1" or app.request_id : "kibana-http-1"` → 2 events

Source	Key fields
Envoy	`interface: http`, `trace_id: kibana-http-1`, `upstream_host: <A's IP>:3000`, `status: 200`, `duration_ms: 3`
Replica A	`app_name: generator-app`, `request_id: kibana-http-1`, `container.name: esd_first_assignment-app-1`, `status: 200`

gRPC: `app.trace_id : "831a082a-0e6d-4eab-af80-d8ab089b6622"` → 2 events

Source	Key fields
Envoy	`interface: grpc`, `upstream_host: <A's IP>:50051`, `grpc_status: OK`, `duration_ms: 55`
Replica A	`app_name: grpc-generator`, `trace_id` (same value), `container.name: esd_first_assignment-app-1`

[Insert the two Kibana screenshots here.]

Reading the results

Client p95 includes a fixed client overhead

Client p95 sits around 200–400 ms in every run, but Envoy's own p95 for the same requests was 3 ms (HTTP) and 14 ms (gRPC), and the replicas' server-side p95 was 1–2.5 ms. The ~200–350 ms gap is the Python client on Windows plus Docker Desktop's port forwarding, and it is present in every run. Client p95 is therefore reported as asked, but differences between runs smaller than this overhead are not treated as real; Envoy's durations are used to explain them.

An environment bottleneck found during the baseline

The first A-only baseline failed 4.7 % of requests with client p95 ≈ 4 s, while A's server-side p95 was under 100 ms. Envoy's durations (p95 ≈ 5 s) placed the delay between Envoy and A, and A's event-loop lag reached 0.8–3.7 s at only ~1.6 % CPU: A's single Node thread was blocked waiting on file I/O, before the request timer started. The cause was SQLite on a Windows bind mount. Moving the database to a Docker named volume removed it (p95 104 ms, 0 failures), and all reported runs use the volume. All replicas still share the same database.

Weighted round robin (D1)

The 2:1:1 weights produced exactly 50 / 25 / 25 %. Latency did not change measurably because A at 10 req/s is still far from its limit. A weight decides who receives traffic; it does not add capacity, so a heavier weight on a replica that is already near its limit would make that replica slower.

Least request vs round robin with a slow replica (D2)

With B delayed by 200 ms, round robin kept sending B a third of all requests, so 33.3 % of requests were slow. Least request sends each new request to the replica with fewer requests in progress, so B received only ~19 % and Envoy's p80 dropped from ~203 ms to 5–7 ms. Envoy's least request compares two randomly chosen replicas, so B is still picked when it is idle or when it is not one of the two compared, which is why its share did not reach zero. Because 19 % is more than 5 %, p95 still falls among the slow requests, so p95 (Envoy and client) stayed near 200 ms in both runs. B stayed "healthy" throughout: it answered `/health` instantly while being slow for real requests.

Failure and recovery (D3)

Health checks: `/health` every **2 s**, **1 s** timeout, **3** failures to remove, **2** successes to restore (gRPC cluster checks port 3000 over HTTP/1). After `docker stop -t 0` on B, every request routed to B failed until Envoy removed it ~9 s later (a first attempt measured ~7 s). The stopped container's IP did not refuse connections, so each failed check waited the full 1 s timeout: about 3 s per check (2 s interval + 1 s timeout), and three checks fit the 6–9 s observed. Errors ≈ traffic to B × detection time ≈ 6.7 req/s × 9 s ≈ 60 per interface (59 measured). While B was out, A and C split traffic 50 / 50 with no errors. After restart, B returned to rotation ~4.5 s later: Node startup plus two passing checks.

Conclusions

1. Strategy for HTTP and for gRPC

For **both** interfaces I would choose **least request with active health checks**.

With healthy, equal replicas, round robin and least request behave the same: traffic spreads evenly (33 / 33 / 33 %) and server-side p95 stays at 1–2.5 ms. The difference appears only when a replica degrades. With B slowed by 200 ms, round robin kept sending it a third of the traffic (33.3 % slow requests), while least request cut that to 18.6–18.8 % and brought p80 from ~203 ms down to 5–7 ms, with no errors in either run. A slow replica also passed every health check in that experiment, so health checks alone would not have protected users; least request did.

Health checks are still needed on top: they are the only thing that removed a **dead** replica. In D3 they took B out of rotation after ~7–9 s and put it back ~4.5 s after it restarted.

For gRPC, the choice only works because Envoy's gRPC listener speaks HTTP/2 and routes **per call**. The client kept a single connection, yet calls were spread 400 / 400 / 400; proxied as plain TCP, every call would have stayed on one replica whatever the policy.

2. A workload where I would choose differently

If each request read data that a replica caches in memory, keyed by product ID, I would use **consistent hashing** (Envoy's ring hash) on the product ID. Requests for the same product would then keep landing on the same replica and hit its warm cache, whereas round robin or least request would spread them and make every replica cache everything. The trade-off is less even load when a few products are much more popular than the rest.

3. What happens if Envoy stops

Envoy is a **single point of failure**. Every HTTP and gRPC request enters through ports 8080 and 8081, so if Envoy stops, clients get connection errors even though A, B and C are healthy. Health checks cannot help, because they run inside Envoy.

Improvement (not implemented): run two Envoy instances in an **active–standby** pair sharing a **floating (virtual) IP**, so the standby takes over the address within seconds if the active one fails. Alternatively, put a managed cloud load balancer in front of several Envoys.